

tf.gradients Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/gradients

Constructs symbolic derivatives of sum of ys w.r.t. x in xs.

Function Signatures

```
tf.gradients( ys, xs, grad_ys=None, name='gradients',
gate_gradients=False, aggregation_method=None,
stop_gradients=None,
unconnected_gradients=tf.UnconnectedGradients.NONE )
```

Code Examples

```
tf.gradients(
ys,
xs,
grad_ys=None,
name='gradients',
gate_gradients=False,
aggregation_method=None,
stop_gradients=None,
unconnected_gradients=tf.UnconnectedGradients.NONE
)
```

```
tf.gradients(  
ys,  
xs,  
grad_ys=None,  
name='gradients',  
gate_gradients=False,  
aggregation_method=None,  
stop_gradients=None,  
unconnected_gradients=tf.UnconnectedGradients.NONE  
)
```

```
tf.UnconnectedGradients.NONE
```

```
@tf.function  
def example():  
    a = tf.constant(0.)  
    b = 2 * a  
    return tf.gradients(a + b, [a, b], stop_gradients=[a, b])  
example()  
[,  
]
```

```
@tf.function
```

```
def example():
```

```
a = tf.constant(0.)
```

Documentation

Constructs symbolic derivatives of sum of ys w.r.t. x in xs.

Used in the notebooks

- Generalized Linear Models

`tf.gradients` is only valid in a graph context. In particular, it is valid in the context of a `tf.function` wrapper, where code is executing as a graph.

`ys` and `xs` are each a Tensor or a list of tensors. `grad_ys` is a list of Tensor, holding the gradients received by the `ys`. The list must be the same length as `ys`.

`gradients()` adds ops to the graph to output the derivatives of `ys` with respect to `xs`. It returns a list of Tensor of length `len(xs)` where each tensor is the $\text{sum}(\text{dy}/\text{dx})$ for `y` in `ys` and for `x` in `xs`.

`grad_ys` is a list of tensors of the same length as `ys` that holds the initial gradients for each `y` in `ys`. When `grad_ys` is `None`, we fill in a tensor of '1's of the shape of `y` for each `y` in `ys`. A user can provide their own initial `grad_ys` to compute the derivatives using a different initial gradient for each `y` (e.g., if one wanted to weight the gradient differently for each value in each `y`).

`stop_gradients` is a Tensor or a list of tensors to be considered constant with respect to all `xs`. These tensors will not be backpropagated through, as though they had been explicitly disconnected using `stop_gradient`. Among other things, this allows computation of partial derivatives as opposed to total derivatives. For example:

Here the partial derivatives `g` evaluate to `[1.0, 1.0]`, compared to the

total derivatives `tf.gradients(a + b, [a, b])`, which take into account the influence of `a` on `b` and evaluate to `[3.0, 1.0]`. Note that the above is equivalent to:

`stop_gradients` provides a way of stopping gradient after the graph has already been constructed, as compared to `tf.stop_gradient` which is used during graph construction. When the two approaches are combined, backpropagation stops at both `tf.stop_gradient` nodes and nodes in `stop_gradients`, whichever is encountered first.

All integer tensors are considered constant with respect to all `xs`, as if they were included in `stop_gradients`.

`unconnected_gradients` determines the value returned for each `x` in `xs` if it is unconnected in the graph to `ys`. By default this is `None` to safeguard against errors. Mathematically these gradients are zero which can be requested using the 'zero' option. `tf.UnconnectedGradients` provides the following options and behaviors:

Let us take one practical example which comes during the back propagation phase. This function is used to evaluate the derivatives of the cost function with respect to Weights `Ws` and Biases `bs`. Below sample implementation provides the explanation of what it is actually used for :

Returns

Raises